

Лабораторная работа №11

Имитационное моделирование

Александрова Ульяна Вадимовна

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Выполнение лабораторной работы	7
3.1	Построение модели с помощью CPNTools	7
3.1.1	1.1.	7
3.1.2	1.2.	8
3.1.3	1.3.	8
3.1.4	Зададим декларации системы	9
3.1.5	Зададим параметры модели на графах сети.	12
3.2	Мониторинг параметров моделируемой системы	15
4	Выводы	23

Список иллюстраций

3.1	Граф системы	7
3.2	Граф генератора	8
3.3	Граф обработки заявок	9
3.4	Декларации модели	10
3.5	Граф системы с заданными параметрами	12
3.6	Граф принятия заявок с заданными параметрами	14
3.7	Граф обработки заявок с заданными параметрами	15
3.8	Функция Остановка	16
3.9	Мониторы	16
3.10	Функция Queue Delay	17
3.11	Queue_Delay.log	18
3.12	График изменения задержки в очереди	18
3.13	Текст программы для вывода графика	19
3.14	Queue Delay Real	19
3.15	Логи Queue Delay Real	20
3.16	Long Delay Time	21
3.17	londelaytime	21
3.18	Периоды времени, когда значения задержки в очереди превышали заданное значение	22

Список таблиц

1 Цель работы

Целью данной лабораторной работы является создание модели системы массового обслуживания $M|M|1$ при помощи утилиты CPNTools.

2 Теоретическое введение

В систему поступает поток заявок двух типов, распределённый по пуассоновскому закону. Заявки поступают в очередь сервера на обработку. Дисциплина очереди FIFO. Если сервер находится в режиме ожидания (нет заявок на сервере), то заявка поступает на обработку сервером.

3 Выполнение лабораторной работы

3.1 Построение модели с помощью CPNTools

Будем использовать три отдельных листа: на первом листе опишем граф системы, на втором — генератор заявок, на третьем — сервер обработки заявок.

3.1.1 1.1.

Сеть имеет 2 позиции (очередь — Queue, обслуженные заявки — Completed) и два перехода (генерировать заявку — Arrivals, передать заявку на обработку серверу — Server). Переходы имеют сложную иерархическую структуру, задаваемую на отдельных листах модели (с помощью соответствующего инструмента меню — Hierarchy) (рис. 3.1).

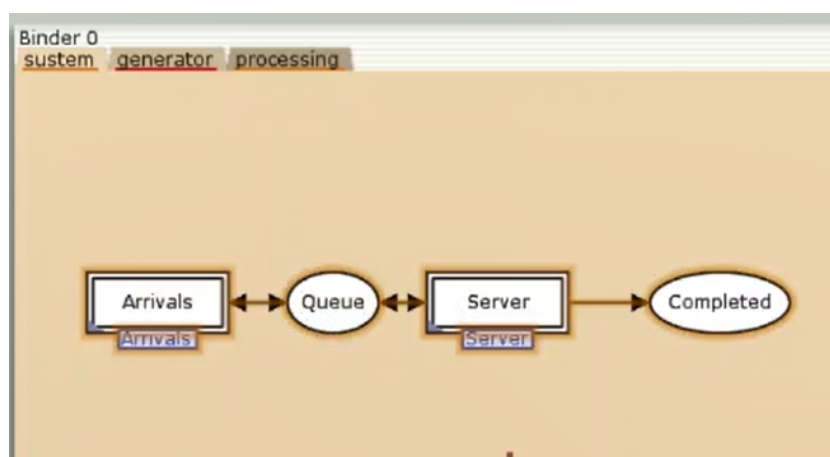


Рис. 3.1: Граф системы

3.1.2 1.2.

Граф генератора заявок имеет 3 позиции (текущая заявка — Init, следующая заявка — Next, очередь — Queue из листа System) и 2 перехода (Init — определяет распределение поступления заявок по экспоненциальному закону с интенсивностью 100 заявок в единицу времени, Arrive — определяет поступление заявок в очередь) (рис. 3.2).

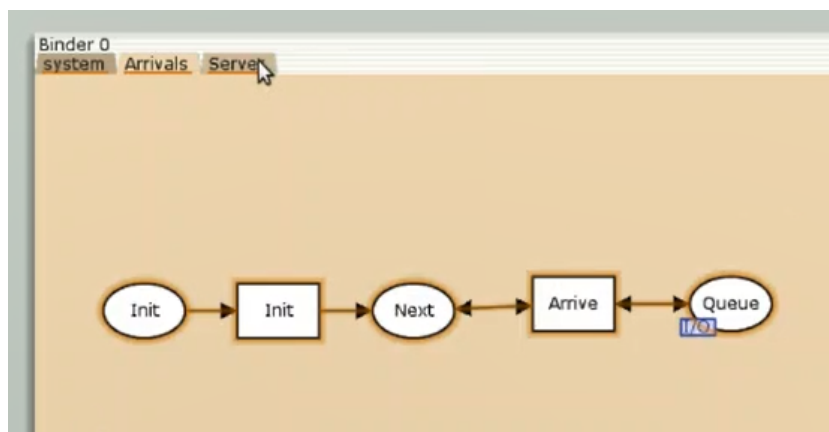


Рис. 3.2: Граф генератора

3.1.3 1.3.

Граф процесса обработки заявок на сервере имеет 4 позиции (Busy — сервер занят, Idle — сервер в режиме ожидания, Queue и Complited из листа System) и 2 перехода (Start — начать обработку заявки, Stop — закончить обработку заявки) (рис. 3.3).

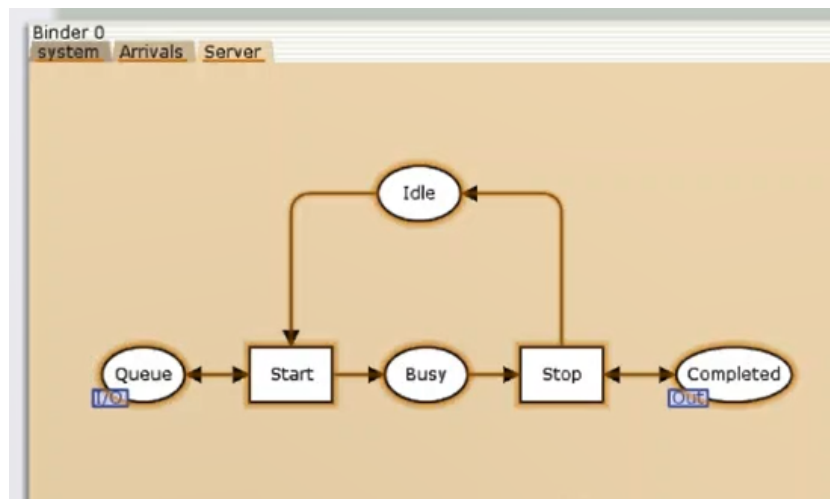


Рис. 3.3: Граф обработки заявок

3.1.4 Зададим декларации системы

Определим множества цветов системы (colorset) (рис. 3.4):

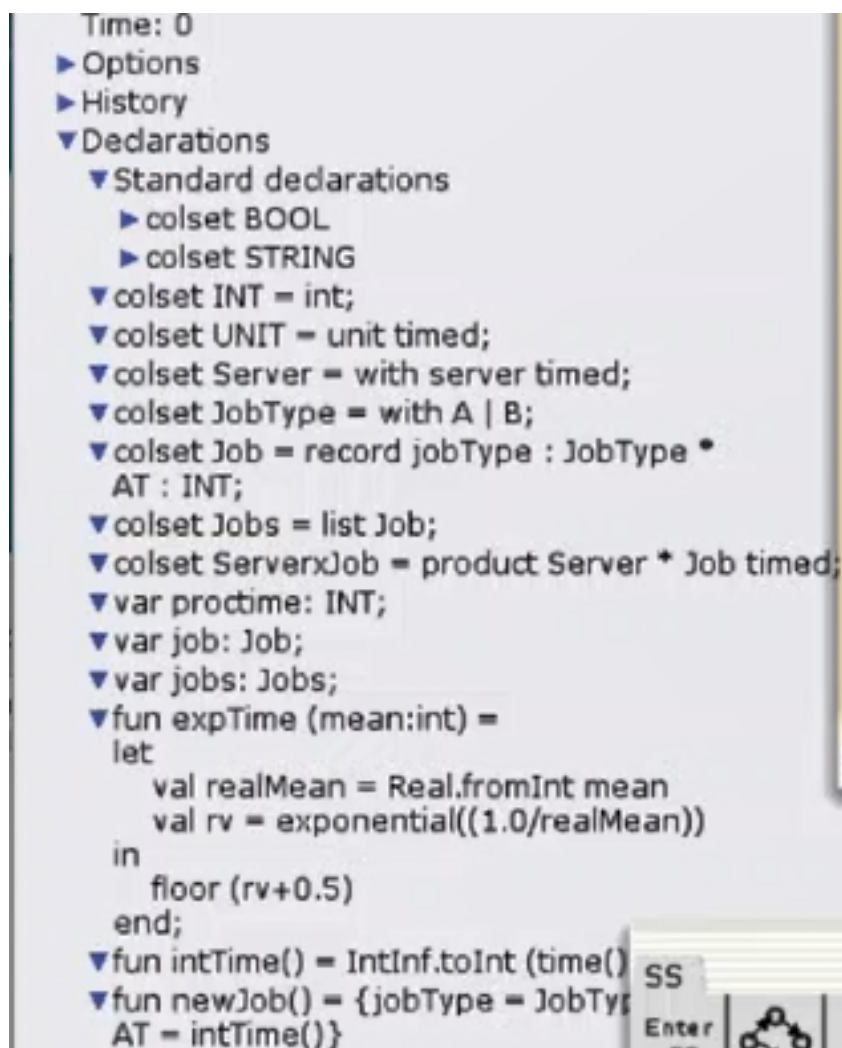


Рис. 3.4: Декларации модели

- фишки типа UNIT определяют моменты времени;
- фишки типа INT определяют моменты поступления заявок в систему.
- фишки типа JobType определяют 2 типа заявок — A и B;
- кортеж Job имеет 2 поля: jobType определяет тип работы (соответственно имеет тип JobType, поле AT имеет тип INT и используется для хранения времени нахождения заявки в системе;

- фишки Jobs — список заявок;
- фишки типа ServerxJob — определяют состояние сервера, занятого обработкой заявок.

```
colset UNIT = unit timed;
colset INT = int;
colset Server = with server timed;
colset JobType = with A | B;
colset Job = record jobType : JobType *
AT : INT;
colset Jobs = list Job;
colset ServerxJob = product Server * Job timed;
```

Переменные модели:

- proctime — определяет время обработки заявки;
- job — определяет тип заявки;
- jobs — определяет поступление заявок в очередь.

```
var proctime : INT;
var job: Job;
var jobs: Jobs;
```

Определим функции системы:

- функция expTime описывает генерацию целочисленных значений через интервалы времени, распределённые по экспоненциальному закону;
- функция intTime преобразует текущее модельное время в целое число;

- функция `newJob` возвращает значение из набора `Job` — случайный выбор типа заявки (A или B).

```
fun expTime (mean: Int) =
let
val realMean = Real.fromInt mean
val rv = exponential((1.0/realMean))
in
floor (rv+0.5)
end;
fun intTime() = IntInf.toInt (time());
fun newJob() = {jobType = JobType.ran(),
AT
= intTime()}
```

3.1.5 Зададим параметры модели на графах сети.

- у позиции `Queue` множество цветов фишек — `Jobs`; начальная маркировка `1'[]` определяет, что изначально очередь пуста.
- у позиции `Completed` множество цветов фишек — `Job` (рис. 3.5).

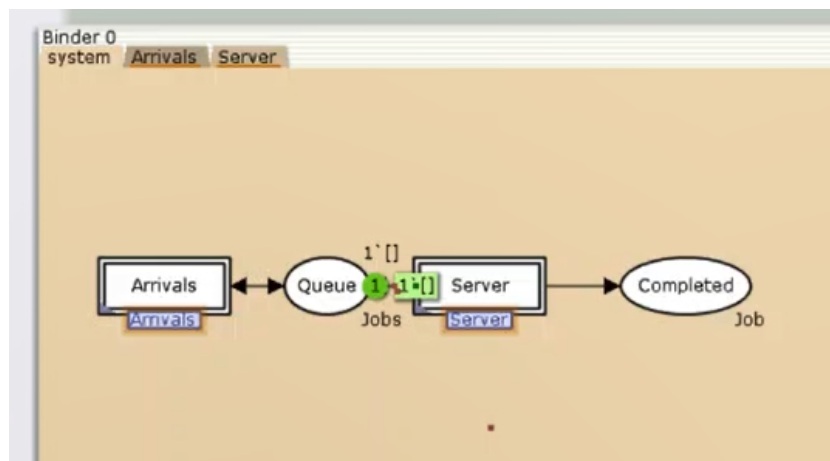


Рис. 3.5: Граф системы с заданными параметрами

- у позиции Init: множество цветов фишек — UNIT; начальная маркировка $1'() [0?]$ определяет, что поступление заявок в систему начинается с нулевого момента времени;
- у позиции Next: множество цветов фишек — UNIT;
- на дуге от позиции Init к переходу Init выражение $()$ задаёт генерацию заявок;
- на дуге от переходов Init и Arrive к позиции Next выражение $()@+expTime(100)$ задаёт экспоненциальное распределение времени между поступлениями заявок;
- на дуге от позиции Next к переходу Arrive выражение $()$ задаёт перемещение фишки;
- на дуге от перехода Arrive к позиции Queue выражение $jobs^{1}$ задает поступление заявки в очередь;
- на дуге от позиции Queue к переходу Arrive выражение $jobs$ задаёт обратную связь (рис. 3.6).

¹job

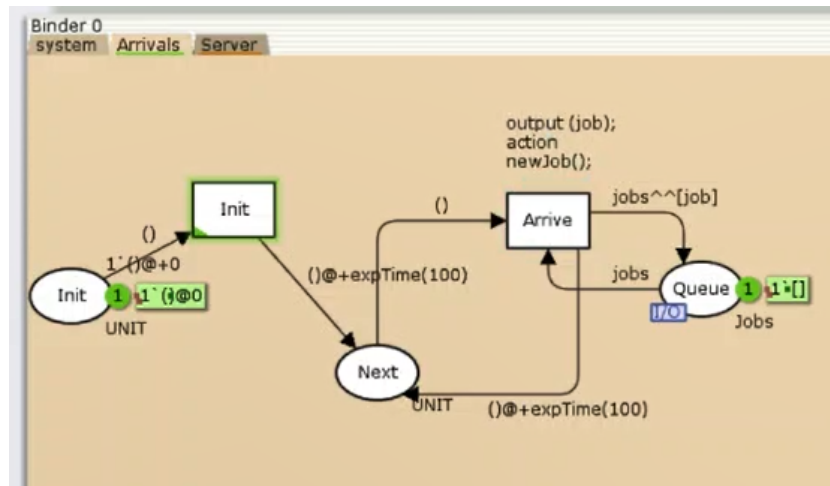


Рис. 3.6: Граф принятия заявок с заданными параметрами

- у позиции Busy: множество цветов фишек — Server, начальное значение маркировки — $1'server@0$ определяет, что изначально на сервере нет заявок на обслуживание;
- у позиции Idle: множество цветов фишек — ServerxJob;
- переход Start имеет сегмент кода `output (proctime); action expTime(90);` определяющий, что время обслуживания заявки распределено по экспоненциальному закону со средним временем обработки в 90 единиц времени;
- на дуге от позиции Queue к переходу Start выражение `job::jobs` определяет, что сервер может начать обработку заявки, если в очереди есть хотя бы одна заявка;
- на дуге от перехода Start к позиции Busy выражение `(server,job)@+proctime` запускает функцию расчёта времени обработки заявки на сервере;
- на дуге от позиции Busy к переходу Stop выражение `(server,job)` говорит о

завершении обработки заявки на сервере;

- на дуге от перехода Stop к позиции Completed выражение job показывает, что заявка считается обслуженной;
- выражение server на дугах от и к позиции Idle определяет изменение состояния сервера (обрабатывает заявки или ожидает);
- на дуге от перехода Start к позиции Queue выражение jobs задаёт обратную связь (рис. 3.7).

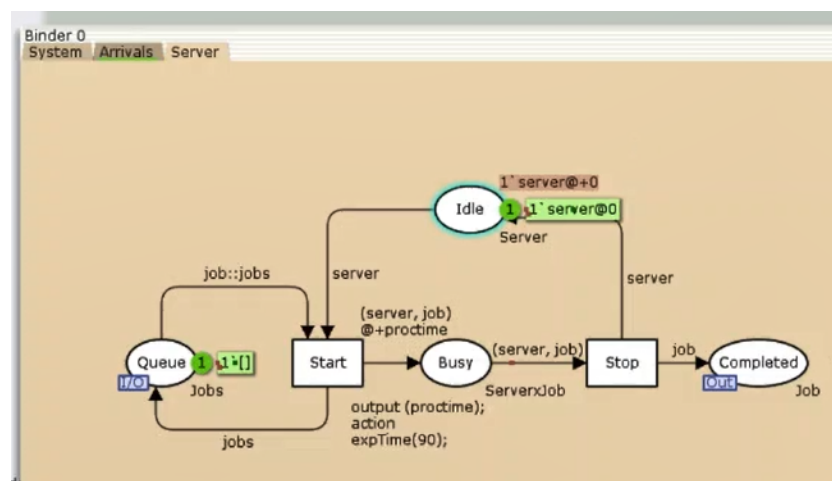


Рис. 3.7: Граф обработки заявок с заданными параметрами

3.2 Мониторинг параметров моделируемой системы

Потребуется палитра Monitoring. Выбираем Break Point (точка останова) и устанавливаем её на переход Start. После этого в разделе меню Monitor появится новый подраздел, который назовём Ostanovka. В этом подразделе необходимо внести изменения в функцию Predicate, которая будет выполняться при запуске монитора (рис. 3.8).

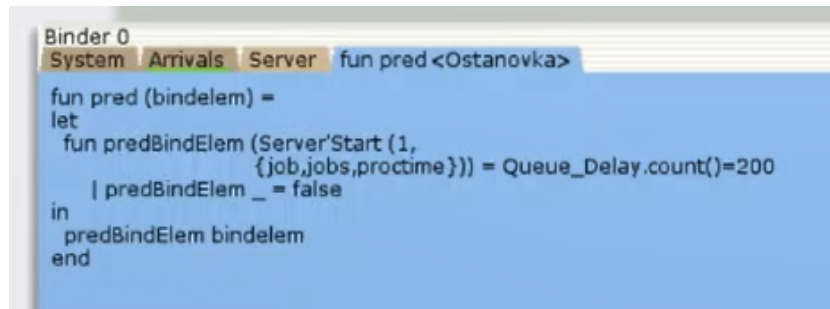


Рис. 3.8: Функция Остановка

```

fun pred (bindelem) =
let
fun predBindElem (Server'Start (1, {job,jobs,proctime}))
= Queue_Delay.count()=200
| predBindElem _ = false
in
predBindElem bindelem
end

```

Необходимо определить конструкцию `Queue_Delay.count()`. С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start.

Появившийся в меню монитор называем Queue Delay (без подчеркивания) (рис. 3.9).

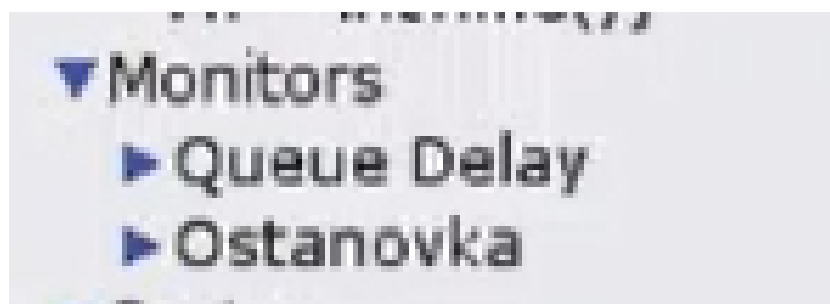
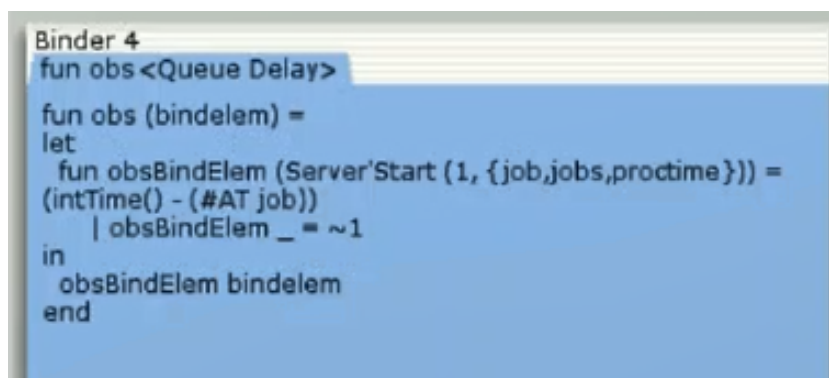


Рис. 3.9: Мониторы

Функция Observer выполняется тогда, когда функция предикатора выдаёт значение true. Изменим её так, чтобы получить значение задержки в очереди

(рис. 3.10).



```
Binder 4
fun obs<Queue Delay>
  fun obs (bindelem) =
    let
      fun obsBindElem (Server'Start (1, {job,jobs,proctime})) =
        (intTime() - (#AT job))
        | obsBindElem _ = ~1
    in
      obsBindElem bindelem
    end
```

Рис. 3.10: Функция Queue Delay

```
fun obs (bindelem) =
let
fun obsBindElem (Server'Start (1, {job, jobs, proctime}))
= (intTime() - (#AT job))
| obsBindElem _ = ~1
in
obsBindElem bindelem
end
```

После запуска программы на выполнение в каталоге с кодом программы появился файл Queue_Delay.log, содержащий в первой колонке — значение задержки очереди, во второй — счётчик, в третьей — шаг, в четвёртой — время (рис. 3.11).

/home/openmodelica/mip/output/logfiles/Queue_Delay.log -			
Файл Правка Поиск Вид Документ Справка			
#data	counter	step	time
0	1	3	51
0	2	6	254
0	3	9	439
0	4	12	478
0	5	15	983
0	6	18	1237
76	7	21	1317
0	8	24	1535
98	9	30	1652
161	10	32	1727
182	11	35	1788
264	12	38	1891
149	13	40	1909
129	14	42	1967

Рис. 3.11: Queue_Delay.log

Преобразую данные в график через GNUplot (рис. 3.12) (рис. 3.13).

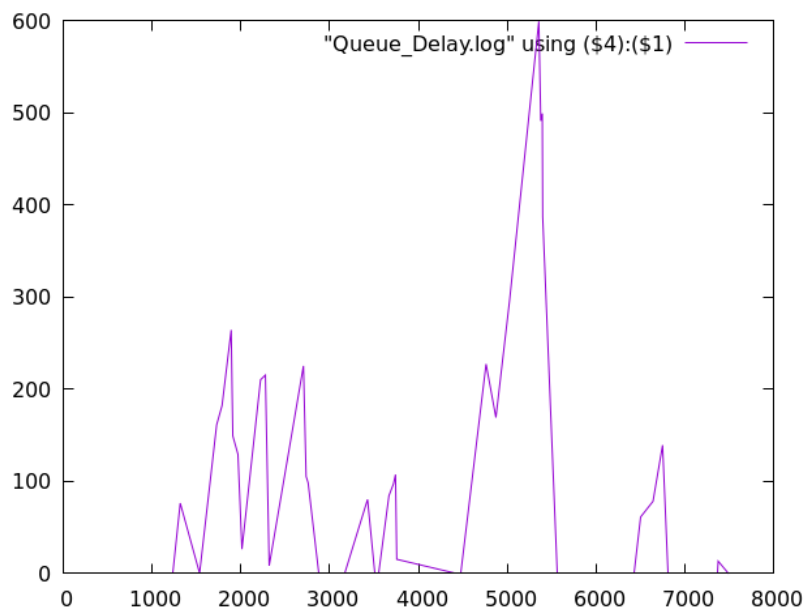
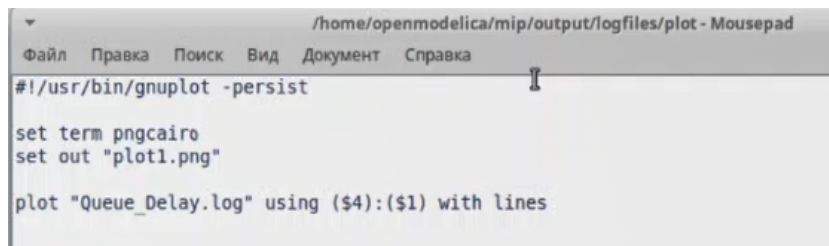


Рис. 3.12: График изменения задержки в очереди

A screenshot of a text editor window titled "/home/openmodelica/mip/output/logfiles/plot - Mousepad". The window has a menu bar with "Файл", "Правка", "Поиск", "Вид", "Документ", and "Справка". The text content is a gnuplot script:

```
#!/usr/bin/gnuplot -persist

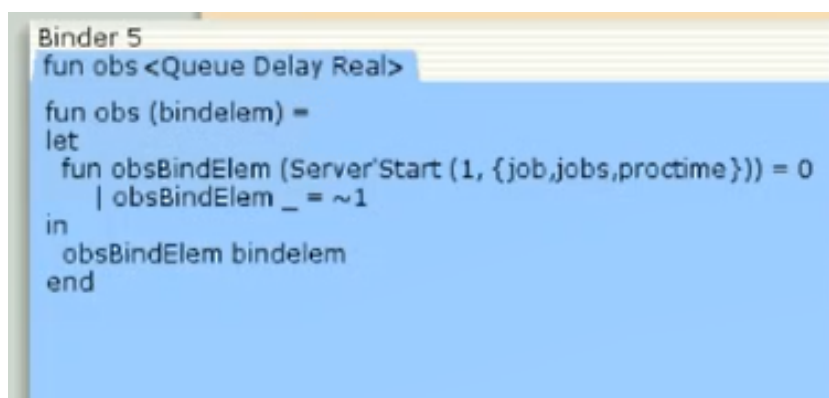
set term pngcairo
set out "plot1.png"

plot "Queue_Delay.log" using ($4):($1) with lines
```

Рис. 3.13: Текст программы для вывода графика

Посчитаем задержку в действительных значениях. С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay Real.

Функцию Observer изменим следующим образом (рис. 3.14):

A screenshot of a code editor window titled "Binder 5". The code is as follows:

```
fun obs <Queue Delay Real>
  fun obs (bindelem) =
    let
      fun obsBindElem (Server'Start (1, {job,jobs,proctime})) = 0
        | obsBindElem _ = ~1
    in
      obsBindElem bindelem
    end
```

Рис. 3.14: Queue Delay Real

```
fun obs (bindelem) =
  let
    fun obsBindElem (Server'Start (1, {job, jobs, proctime}))
      = Real.fromInt(intTime() - (#AT job))
      | obsBindElem _ = ~1.0
    in
      obsBindElem bindelem
    end
```

По сравнению с предыдущим описанием функции добавлено преобразование

значения функции из целого в действительное, при этом `obsBindElem _` принимает значение ~ 1.0 .

После запуска программы на выполнение в каталоге с кодом программы появится файл `Queue_Delay_Real.log` с содержимым, аналогичным содержимому файла `Queue_Delay.log`, но значения задержки имеют действительный тип (рис. 3.15).

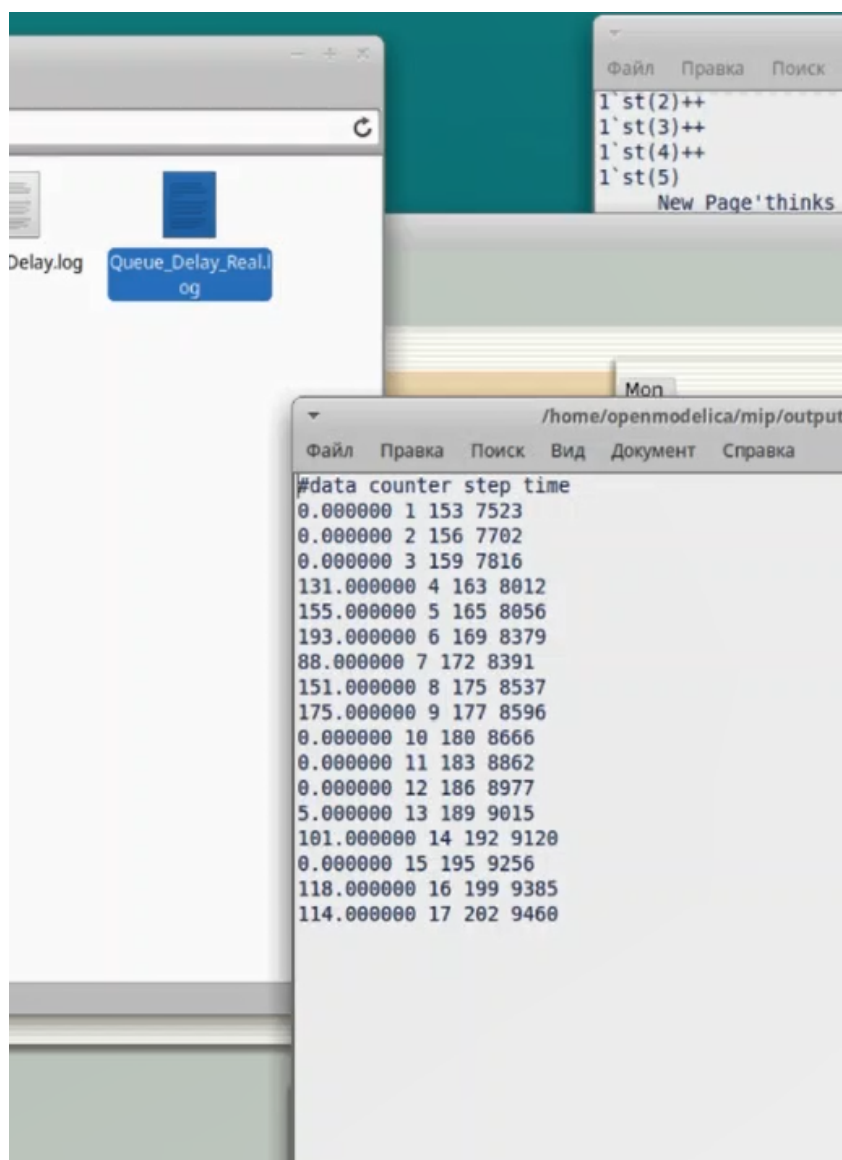


Рис. 3.15: Логи Queue Delay Real

Посчитаем, сколько раз задержка превысила заданное значение. С помощью

палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Монитор называем Long Delay Time.

Функцию Observer изменим следующим образом (рис. 3.16):

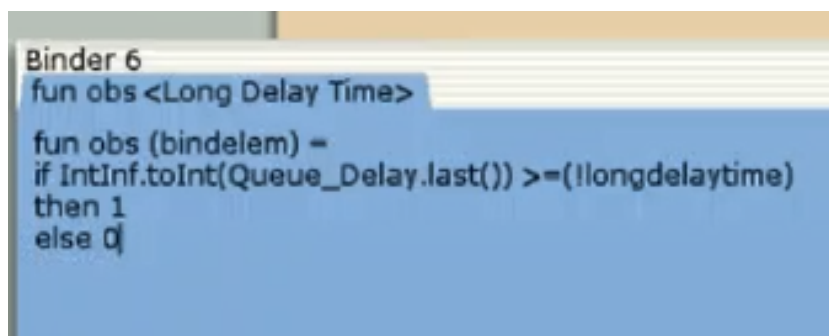


Рис. 3.16: Long Delay Time

```
fun obs (bindelem) =
if IntInf.toInt(Queue_Delay.last())>=(!longdelaytime)
then 1
else 0
```

Если значение монитора Queue Delay превысит некоторое заданное значение, то функция выдаст 1, в противном случае — 0. Восклицательный знак означает разыменование ссылки. При этом необходимо в декларациях (рис. 11.13) задать глобальную переменную (в форме ссылки на число 200) (рис. 3.17):



Рис. 3.17: londelaytime

```
globref longdelaytime = 200;
```

Строю график (рис. 3.18).

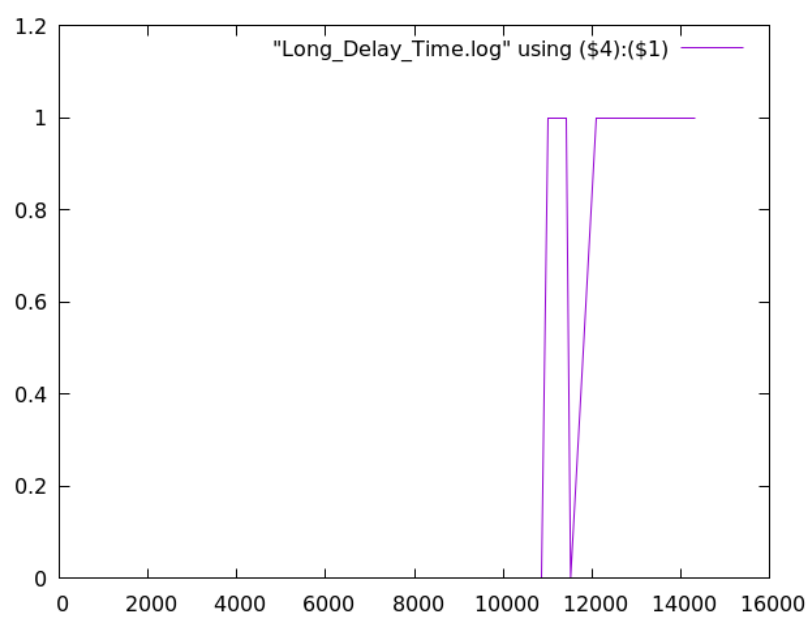


Рис. 3.18: Периоды времени, когда значения задержки в очереди превышали заданное значение

4 Выводы

В ходе выполнения данной лабораторной работы я построила модель системы массового обслуживания $M|M|1$ при помощи утилиты CPNTools.